PROGRAMMING & CS FUNDAMENTALS

Hash Tables Visual Guide

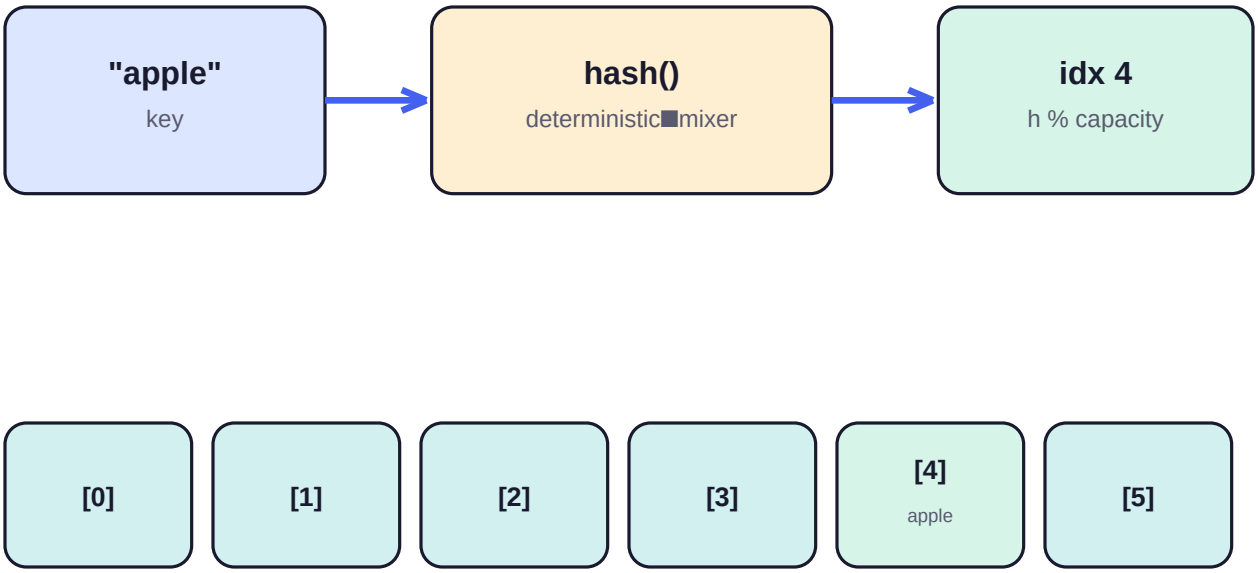
Average O(1) lookup —
the engine inside dicts, sets, caches, and indexes.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Pipeline: Key -> Hash -> Index

A hash function turns any key into an array slot.



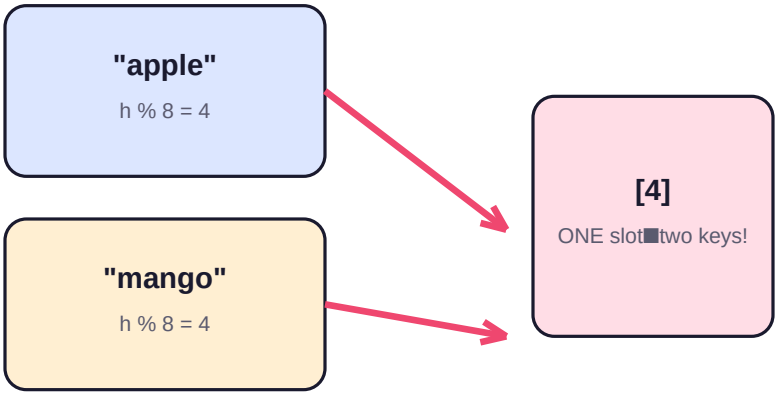
index = hash(key) mod capacity. Same key -> same hash -> same slot, every time.

- Good hash: deterministic, fast, uniform — spreads keys evenly across slots.
- Modulo maps the huge hash space down to table capacity.
- Keys must be hashable AND immutable (strings, tuples) — mutating a key corrupts lookup.

STEP 2/6

Collisions Are Inevitable

Pigeonhole principle: more possible keys than slots.



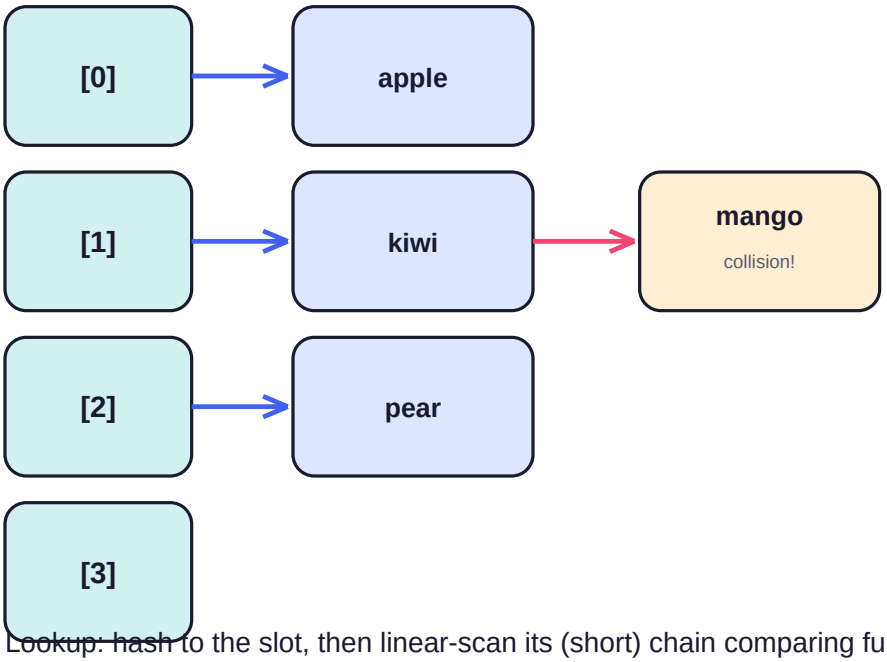
- Birthday paradox: collisions appear far earlier than intuition says (~sqrt(capacity) keys).
- A hash table is a collision-resolution strategy with a hash function attached.
- Two families: chaining (external) vs open addressing (in-place probing).

Worst case with bad collisions: O(n) per op — a degenerate table is a linked list wearing a costume.

STEP 3/6

Chaining: Buckets of Lists

Each slot holds a chain — collisions just lengthen it.



Lookup: hash to the slot, then linear-scan its (short) chain comparing full keys.

- Simple, deletion-friendly (just unlink), never 'full' — but pays pointer overhead.
- Java HashMap, Python's old dict design family: chains (plus tree-ification past 8).
- Long chains signal a bad hash function or an overloaded table — resize, don't suffer.

STEP 4/6

Open Addressing: Probe for Home

Everything lives IN the array — collisions probe neighboring slots.

mango wanted [2] (taken) -> probed [3]

```
linear:    i, i+1, i+2, ...      # simple, clusters badly
quadratic: i, i+1, i+4, i+9 ...  # spreads, needs care
double:    i + k*h2(key) ...     # second hash as step
```

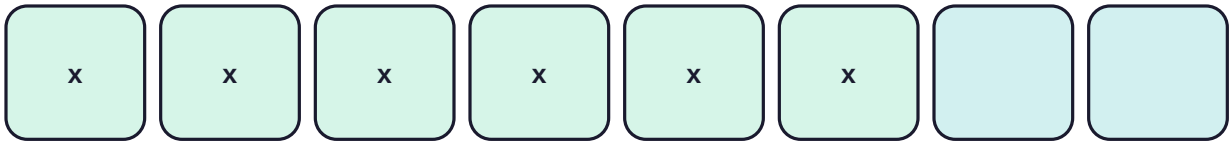
- Deletion needs tombstones: a bare empty slot would break probe chains.
- Cache-friendly (one flat array) — why Python dicts and Swiss tables use it.
- Primary clustering: linear probing clumps fill into long runs — keep load low.

STEP 5/6

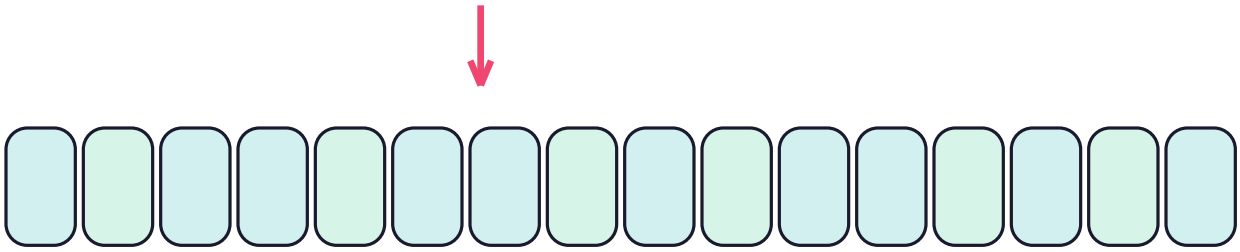
Load Factor & Resizing

Fullness is the enemy — resize BEFORE performance collapses.

load factor a = n / capacity



a = 6/8 = 0.75 -> RESIZE: allocate 16, rehash every key



new capacity 16, a back to 0.375 — keys land on fresh slots via rehash

- Resize at a ~ 0.7 (chaining) or ~ 0.5 (open addressing) — probing degrades fast past half full.
- Resize = allocate bigger + rehash ALL keys (indices change with capacity): O(n), amortized.
- Python dict: grows at 2/3 full; over-allocates to keep resizes rare.

STEP 6/6

Hash Table Pitfalls

Fast on average — fragile at the edges.

- Mutable keys: hash computed at insert, key mutated later -> entry is lost forever.
- Bad hash (e.g. constant): every op degrades to O(n) — hash-flooding DoS attacks exploit this.
- Iteration order is NOT insertion order (unless the table guarantees it, like Python 3.7+ dicts).
- Floating-point keys: 0.1+0.2 != 0.3 — equality and hashing must agree, or lookups miss.
- Resizing pauses: latency-sensitive systems pre-size tables or resize incrementally.
- Custom objects: define BOTH __hash__ and __eq__ consistently — equal objects must hash equal.

STEP ✓I✓

Key Takeaways

Hash Tables — Remember

- Pipeline: hash(key) mod capacity -> slot; same key always lands together.
- Collisions are certain: chain them (buckets of lists) or probe (linear/quadratic/double).
- Watch the load factor: resize (allocate + rehash all) around 0.7 chaining / 0.5 probing.
- Average O(1), worst O(n) — a bad hash function turns your table into a linked list.
- Keys must be immutable and hash-consistent with equality.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev